

Text Processing für NLP

Contents

- Bag of Words für Text-Klassifikation
- Bag of Words
- Wichtige Konzepte im NLP
- Text Processing Pipeline
- Ähnlichkeit von Texten
- Statistische Text-Generierung

Lernziele

- Bag of Words als einfachen Ansatz der Textverarbeitung für Klassifikationsmodelle kennenlernen
- Wichtige Konzepte des NLP wie Korpus, Vokabular, Tokenisierung, Stopword Removal, Stemming und Lemmatisierung verstehen
- Konzepte für Berechnung semantischer Ähnlichkeit von Texten kennenlernen
- Mit einem einfachen statistischen Ansatz (Shannon's Methode) Texte generieren

Bag of Words für Text-Klassifikation

Beispiel: Sentiment-Klassifikation

- **Sentiment:** emotionaler Zustand, welcher in einem Text zum Ausdruck gebracht wird.
- einfachste Aufgabe: Klassifiziere die Haltung in einem Text als **positiv**, **negativ** oder **neutral**.
- weiterführend: Zustände wie "sauer", "glücklich" oder "wütend" erkennen und einen Text entsprechend klassifizieren

Diskussion (3–5 Min.)

Wie könnte ein Algorithmus zur Vorhersage des Sentiments (positiv/negativ/neutral) aussehen?

```
import pandas as pd
%matplotlib inline
```

```
df = pd.read_csv("movies.csv")
```

```
df.head()
```

	review	sentiment
0	arnold schwarzenegger has been an icon for act...	0
1	good films are hard to find these days . great...	1
2	quaid stars as a man who has taken up the prof...	1
3	we could paraphrase michelle pfieffer's charac...	0
4	kolya is one of the richest films i've seen in...	1

```
df[df.sentiment==1].sample(1, random_state=23)["review"].values[0]
```

" a breed apart " casts rutger hauer as a crazy , bird-loving recluse who picks his feathered friend

The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!

15



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

Bag of Words

- **Feature-Space ermitteln:** Alle Wörter, die in den Trainings-Dokumenten vorkommen, extrahieren. Jedes Wort ist ein **Feature**. Die Features bilden gemeinsam das **Vokabular**.
- **Feature-Repräsentation:** Pro Trainingsdokument die Anzahl der Vorkommnisse jedes Wortes aus dem Vokabular zählen → Feature-Repräsentation pro Dokument ist ein Vektor der Länge = Größe des Vokabulars, mit den Zählungen der Wörter in dem Dokument
- **Anwendung:** Trainiere einen Klassifikator (z.B. logistische Regression, Naive Bayes, k-NN) auf den Feature-Vektoren der Trainingsdokumente
- **Vorhersage:** Für ein neues Dokument die Feature-Repräsentation berechnen. Dabei die sog. **out of vocabulary** Wörter ignorieren. Anschließend den trainierten Klassifikator darauf anwenden
- **Implementierung:** Es gibt fertige Implementierungen in Python (z.B. `CountVectorizer` in `scikit-learn`)

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
vec = CountVectorizer(min_df=2)
```

`CountVectorizer()`

- Splittet einen String in Wörter und zählt deren Vorkommnisse
- Hat mehrere Parameter, die man dem Konstruktor mitgeben kann; z.B. `min_df=2` führt dazu, dass nur Wörter, die mind. 2 Mal vorkommen, betrachtet werden (reduziert die Vokabular-Größe)
- Weitere interessante Parameter: Akzente ignorieren (`strip_accents`, Teil der sog. Zeichen-Normalisierung), alles kleingeschrieben (`lowercase`), Art der Wort-Trennung (`tokenizer`), usw.
- Hat wie ML Modell-Klassen oder der `OneHotEncoder` eine Methode `fit()`. Diese implementiert das eigentliche BoW-Verfahren. Danach sind die Wörter und ihre Anzahlen berechnet und in dem Objekt gespeichert.

- Methode `transform()` wird nach `fit()` ausgeführt und gibt die Dokumente in die berechnete Feature-Repräsentation zurück.
- Methode `fit_transform()` macht beides in einem
- Es gibt noch einige Methoden mehr, z.B. `inverse_transform()`. Siehe Dokumentation.

```
# erstmal nur die ersten 5 Reviews
movie_counts = vec.fit_transform(df[:5]["review"])
```

```
# Methode `get_feature_names_out()` listet alle Features (Wörter) auf
vec.get_feature_names_out()
```

```
array(['about', 'acting', 'action', 'after', 'all', 'also', 'amazing',
      'american', 'an', 'and', 'another', 'any', 'appealing', 'are',
      'as', 'at', 'be', 'because', 'been', 'between', 'beyond', 'but',
      'by', 'can', 'casting', 'character', 'characters', 'come',
      'consider', 'could', 'days', 'decide', 'devil', 'director', 'do',
      'does', 'doesn', 'don', 'ends', 'enough', 'even', 'face', 'feel',
      'film', 'films', 'first', 'for', 'foreign', 'get', 'getting',
      'going', 'good', 'hard', 'has', 'have', 'he', 'help', 'her', 'him',
      'his', 'how', 'husband', 'if', 'in', 'instead', 'into', 'is', 'it',
      'know', 'let', 'life', 'like', 'long', 'made', 'make', 'makes',
      'man', 'mass', 'matter', 'maybe', 'million', 'money', 'more',
      'movie', 'much', 'no', 'not', 'of', 'old', 'on', 'one', 'or',
      'out', 'parts', 'performance', 'plot', 'provides', 'really',
      'right', 'sacrifices', 'say', 'scene', 'school', 'script', 'see',
      'seen', 'should', 'since', 'situations', 'so', 'soldiers', 'some',
      'son', 'still', 'story', 'strong', 'take', 'team', 'than', 'that',
      'the', 'their', 'there', 'these', 'they', 'things', 'thinking',
      'this', 'though', 'thriller', 'time', 'to', 'true', 'two', 'up',
      'upon', 'us', 'very', 'walked', 'was', 'we', 'well', 'what',
      'when', 'where', 'which', 'while', 'who', 'why', 'will', 'with',
      'without', 'woman', 'won', 'work', 'works', 'world', 'would',
      'wrong', 'year', 'you'], dtype=object)
```

```
# 5 reviews in numerischer Feature Repräsentation nach BoW  
# jede Zeile repräsentiert ein Review  
# jede Spalte repräsentiert eines der Wörter aus allen Reviews  
# es gibt Nullen, da nicht jedes Wort in jedem Review vorkommt  
movie_counts.toarray()
```

```

array([[ 0,  0,  5,  0,  2,  1,  0,  0,  4,  9,  2,  1,  0,  4,  2,  4,
        2,  1,  2,  0,  0, 10,  0,  3,  0,  1,  0,  1,  1,  1,  1,  0,
        6,  0,  2,  1,  2,  2,  0,  2,  2,  0,  1,  8,  2,  0,  5,  0,
        1,  1,  1,  0,  1,  5,  3,  8,  2,  0,  2,  3,  1,  0,  2,  7,
        1,  2,  5, 18,  1,  2,  0,  6,  1,  2,  1,  1,  2,  0,  1,  1,
        1,  1,  1,  4,  2,  1,  2,  9,  0,  0,  4,  1,  0,  1,  1,  0,
        0,  1,  2,  0,  1,  0,  0,  1,  4,  0,  1,  1,  0,  4,  0,  3,
        0,  1,  0,  0,  1,  0,  1,  7, 28,  0,  2,  3,  3,  1,  0,  7,
        0,  1,  0, 14,  1,  0,  0,  1,  1,  1,  1,  8,  1,  1,  1,  4,
        3,  2,  0,  1,  1,  1,  4,  0,  1,  0,  0,  0,  4,  1,  0,  0,
        5]),
 [ 1,  3,  2,  0,  3,  0,  1,  1,  2, 23,  1,  0,  0,  3,  4,  2,
   0,  0,  1,  3,  1,  3,  4,  0,  1,  4,  1,  0,  1,  0,  2,  1,
   1,  1,  1,  0,  0,  0,  1,  1,  0,  1,  0,  8,  2,  0,  5,  0,
   1,  1,  0,  1,  1,  1,  0,  0,  1,  2,  2,  3,  0,  1,  0, 11,
   0,  1,  8,  3,  0,  1,  3,  0,  1,  0,  1,  1,  1,  1,  0,  0,
   1,  1,  0,  3,  1,  2,  1, 21,  1,  2,  4,  0,  1,  0,  1,  0,
   1,  0,  1,  2,  0,  0,  1,  2,  0,  1,  0,  1,  2,  0,  2,  0,
   0,  0,  4,  1,  1,  1,  0,  5, 40,  1,  0,  3,  1,  1,  1,  2,
   0,  1,  0,  8,  1,  1,  3,  1,  0,  1,  0,  2,  0,  2,  0,  1,
   0,  2,  2,  1,  0,  1,  8,  0,  1,  0,  1,  1,  1,  0,  1,  1,
   1]),
 [ 1,  0,  0,  1,  1,  0,  1,  0,  1,  7,  0,  1,  1,  4,  4,  3,
   3,  2,  0,  1,  0,  4,  2,  1,  0,  0,  0,  1,  0,  1,  0,  1,
   0,  0,  0,  1,  0,  0,  0,  0,  0,  1,  0,  0,  0,  1,  2,  0,
   1,  0,  3,  0,  2,  2,  0,  7,  0,  1,  0,  1,  0,  0,  2,  6,
   1,  1, 11,  3,  1,  0,  0,  0,  0,  0,  0,  1,  1,  0,  0,  0,
   0,  1,  1,  5,  0,  0,  1,  5,  0,  1,  0,  0,  2,  1,  0,  1,
   1,  1,  1,  1,  0,  1,  0,  1,  1,  0,  0,  1,  0,  2,  0,  1,
   0,  0,  0,  1,  1,  1,  1,  1, 20,  1,  3,  0,  3,  0,  0,  3,
   0,  0,  1,  9,  0,  1,  2,  0,  1,  1,  0,  1,  0,  0,  1,  1,
   2,  2,  0,  2,  0,  4,  3,  1,  0,  1,  1,  1,  0,  0,  1,  0,
   7]),
 [ 1,  0,  0,  2,  3,  1,  0,  0,  2,  7,  0,  1,  1,  1,  5,  0,
   4,  0,  0,  0,  4,  1,  2,  4,  1,  2,  2,  0,  0,  1,  0,  0,
   0,  1,  3,  1,  1,  1,  0,  3,  1,  0,  1,  1,  1,  1,  2,  2,
   1,  0,  0,  1,  0,  0,  1,  3,  0,  2,  0,  0,  1,  1,  0,  6,
   0,  0,  4,  2,  1,  0,  0,  2,  0,  1,  1,  0,  0,  1,  1,  1,
   0,  0,  0,  1,  1,  1,  1, 11,  0,  1,  0,  4,  0,  0,  1,  1,
   0,  0,  2,  0,  2,  1,  1,  0,  0,  0,  1,  0,  1,  0,  1,  1,

```

```

1, 1, 2, 0, 0, 0, 1, 6, 25, 1, 1, 0, 0, 0, 1, 1,
2, 0, 0, 7, 1, 0, 0, 0, 0, 1, 0, 1, 4, 0, 1, 0,
0, 0, 0, 0, 1, 0, 3, 1, 0, 0, 1, 0, 0, 1, 0, 0,
1],
[ 0, 1, 0, 1, 0, 1, 0, 1, 1, 5, 0, 0, 0, 1, 3, 0,
0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0,
0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1,
0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 2, 0, 0, 1, 2,
0, 0, 1, 2, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 0, 0, 0, 1, 2, 2, 0, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1,
2, 0, 0, 0, 0, 0, 1, 1, 4, 0, 0, 1, 0, 0, 0, 1,
1, 0, 2, 2, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
0, 0, 1, 2, 0, 0, 2, 0, 0, 1, 0, 0, 0, 0, 0, 1,
0]])

```

Wir sehen einige Nullen → Matrix ist leicht *sparse*

Wie groß wird das Vokabular, wenn wir alle Reviews betrachten statt nur der ersten 5?

```

# Größe des Vokabulars zu 5 Reviews
len(vec.get_feature_names_out())
# movie_counts.shape

```

161

```

# nun mit allen 2000 Reviews
movie_counts = vec.fit_transform(df["review"])

```

```

# Größe des Vokabulars zu allen 2000 Reviews
len(vec.get_feature_names_out())

```


24087

- 2000 Dokumente führen zu ca. 24.000 verschiedenen Wörtern. Der Feature-Raum X hätte damit ca. 25.000 Dimensionen!
- Aber: wir haben immerhin *eine numerische* Repräsentation und können einen Klassifikator trainieren.

```
# Das Wort 'actor' ist im Vokabular mit Index 478  
vec.vocabulary_.get('actor')
```

478

```
# Arnold Schwarzenegger ist auch dabei ...  
vec.vocabulary_.get('schwarzenegger')
```

18664

Wichtige Konzepte im NLP

Korpus: eine maschinenlesbare Sammlung von geschriebener oder gesprochener Sprache. Beispiel: Der Brown-Korpus ist eine Sammlung Auszügen aus 500 geschriebenen englischsprachigen Texten verschiedener Genres (Zeitungen, Belletristik, Sachbücher, akademische Texte usw.), die 1963-64 an der Brown-Universität zusammengestellt wurden.

Es gibt verschiedene Dimensionen, um die **Variation** von Korpora zu beschreiben:

- **Sprache** (es gibt >7,000 weltweit). Die meisten NLP Tools werden für die offiziellen Sprachen der industrialisierten Länder entwickelt (Englisch, Chinesisch, Spanisch, Arabisch, etc.). Manchmal gibt es Sprachen-Mix in einer Kommunikation (=code switching).
- **Dialekte**
- **Genre**: Nachrichten, Bücher (Belletristik, Sachbücher, ...), wissenschaftliche Artikel, Filme, Telefon-Konversationen,...
- **Zeitraum**
- Text spiegelt typischerweise **demographische und soziale Aspekte** der Autor*innen / Sprecher*innen wider: Alter, Gender, sozialer / materieller Hintergrund, ...

Sprache ist situiert: wenn wir NLP Modelle entwickeln, müssen wir uns fragen, wer die Sprache / Korpora produziert hat, in welchem Kontext, für welchen Zweck, und sicherstellen, dass die trainierten Modelle zu den Anwendungsdaten passen.

Wörter: Es kommt darauf an, was wir zählen wollen ...

- Wollen wir Satzzeichen als Wörter zählen? Die Interpunktion ist entscheidend, um Grenzen von Dingen zu finden (Kommas, Punkte, Doppelpunkte) und um einige Aspekte der Bedeutung zu identifizieren (Fragezeichen, Ausrufezeichen, Anführungszeichen). Für Klassifikation von Inhalten von Texten sind sie meist irrelevant aber sie spielen eine wichtige Rolle bei der Autoren-Identifikation, da sehr personenspezifisch.
- Was ist mit **New York**? Soll das ein Wort sein? (Frage nach Phrasen, Wortgrenzen...)
- Wollen wir zwischen Groß- und Kleinschreibung unterscheiden? Für Spracherkennung ist das egal, für Part-of-Speech oder Named-Entity Tagging kann Großschreibung wichtig sein (**Das ist eine frische Tomate** vs. **Die Frische der Tomaten**).
- Und was ist mit **Tasse** vs. **Tassen** oder **esse** vs. **isst**? Welche Rolle sollen Morpheme spielen?
- Und was, wenn wir nur an bestimmten Text-Mustern interessiert sind, z.B. Nomen, Abkürzungen, Email-Adressen?

Token (=Vorkommnis) und **Type** (=Typ): Unterscheidung zwischen einem einzelnen Vorkommnis und dem allgemeinen Vorkommnistyp.

- Beispiel: `Für mich. Für uns. Für morgen.`: 6 Token und 4 Types (wir ignorieren Interpunktionen).
- Beispiel: Wie viele Ziffern befinden sich in der Reihe `2200999`? 7 Token (Vorkommnisse) oder 3 Typen drei, `2`, `0` und `9`.

Vokabular: Sammlung aller Wörter in einem Korpus. Alle Wörter werden einfach gelistet, d.h. die Anzahl der Typen entspricht der Größe des Vokabulars.

Corpus	Tokens = N	Types = $ V $
Shakespeare	884 thousand	31 thousand
Brown corpus	1 million	38 thousand
Switchboard telephone conversations	2.4 million	20 thousand
COCA	440 million	2 million
Google N-grams	1 trillion	13 million

Größe des Vokabulars in Abhängigkeit der Korpus-Größe. Jurafsky & Martin, Speech and Language Processing (2026), Chapter 3

- Typischerweise: je größer der Korpus desto mehr Typen gibt es.
- Beziehung zwischen beiden Größen heißt *Herdan's Law* (Linguistik) oder *Heap's Law* (Information Retrieval):

$$|V| = kN^\beta$$

wobei $k > 0$ und $0 < \beta < 1$. k, β hängen vom Korpus und Genre ab aber für die großen Korpora typischerweise $0.67 < \beta < 0.75$.

Text Processing Pipeline

- Verschiedene Verarbeitungsschritte können nützlich sein, um Types, die gleiche Inhalte repräsentieren, zusammenzuführen. Dies reduziert zudem das Vokabular.
- Häufige Schritte:
 1. Bereinigung (Cleaning), z.B. Entfernen von HTML-Tags, URLs, Emojis, ...
 2. Tokenisierung, d.h. Segmentierung von Text in Wörter / Token
 3. Normalisierung von Wort-Formaten, z.B. Kleinschreibung
 4. Entfernen von Stop-Wörtern
 5. Stemming / Lemmatisierung

Tokenisierung (tokenization)

Segmentierung von Fließtext in bedeutsame Segmente: Token

- Trennzeichen: häufig Leerzeichen oder Whitespaces allgemein, oder auch Satzzeichen
- Satzzeichen: werden für viele Anwendungen als separate Token behalten, da sie wichtige Information für Parser tragen (z.B. Satzende).
- Satzzeichen "im Wort" sollen manchmal drin bleiben, z.B. bei `"Ph.D"` oder `"can't"`
- Spezielle Muster wie Datum oder Preis sollen i.d.R. zusammen bleiben, z.B. `"$45.30"` oder `"20.7.2020"` oder `"helena.mihaljevic@htw-berlin.de"` oder `"https://www.python.org/"`.
- Besonderheit bei Zahlen: Komma und Punkt in Zahlen haben andere Bedeutung als im Satz - und je nach Land trennen sie Tausender oder den Dezimalbereich!
- `"gibt's doch gar nicht"` → `"gibt es doch gar nicht"`
- Multi-Wort Ausdrücke: `"New York"` oder `"Rock'n'Roll"` → hierfür muss aber bekannt sein, dass es sich um sog.

benannte Entitäten handelt

Entscheidung hängt ab von Sprache, Aufgabe, etc.

- Tokenisierung kommt häufig ganz am Anfang einer Text Processing Pipeline vor → soll möglichst schnell sein und ist typischerweise deterministisch
- Ein häufig verwendeter Standard basiert auf **regulären Ausdrücken**, z.B., Penn Treebank Tokenizer:
 - split standard contractions, e.g. `don't` -> `do n't` and `they'll` -> `they 'll`
 - treat most punctuation characters as separate tokens
 - split off commas and single quotes, when followed by whitespace
 - separate periods that appear at the end of line

```
from nltk.tokenize import TreebankWordTokenizer
```

```
s1 = '''Good muffins cost $3.88\nin New York.  Please buy me\ntwo of them.\nThanks.'''  
s2 = "They'll save and invest more."  
s3 = "hi, my name can't hello,"
```

```
print(TreebankWordTokenizer().tokenize(s1))
```

```
['Good', 'muffins', 'cost', '$', '3.88', 'in', 'New', 'York.', 'Please', 'buy', 'me', 'two', 'of', 't
```

```
print(TreebankWordTokenizer().tokenize(s2))
```

```
['They', '"ll", 'save', 'and', 'invest', 'more', '.']
```

```
print(TreebankWordTokenizer().tokenize(s3))
```

```
['hi', ',', 'my', 'name', 'ca', "n't", 'hello', ',']
```

Stopword Removal

- Es gibt keine einheitliche Definition von Stop-Wörtern
- Grundsätzlich sind dies Wörter, die aus dem Vokabular entfernt werden können, ohne die Modellierung zu verschlechtern
- Vorteil: Reduktion des Vokabulars → Reduktion der Dimension vom Feature-Space X → kürzere Rechenzeit und einige alg. Verfahren funktionieren besser
- Ansatz 1: Wörter, die häufig in (fast) allen Dokumenten vorkommen
- Ansatz 2: Wörter, die in dem gegebenen Kontext keine echte Bedeutung haben (z.B. `der` oder `um` bei Textklassifikation)

In beiden Fällen nehmen wir an, dass die entfernten Wörter kaum zum Task beitragen

- Achtung: Es hängt stark vom Kontext ab, ob bestimmte Wörter relevant sind und eine Bedeutung haben oder nicht.
- Obwohl es Listen von stop words gibt für verschiedene Sprachen (meistens Teil von Libraries wie `nltk` oder `spaCy`), verbessert sich die Performanz eines Modells durch die Entfernung von stop words eher nicht (außer teils bei Text-Klassifikation)

- Sie können diesen Schritt in der Pipeline als einen Hyperparameter betrachten und evaluieren auf Ihren Validierungsdaten.

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
print(len(ENGLISH_STOP_WORDS))
print(list(ENGLISH_STOP_WORDS)[:50])
```

318

['very', 'beside', 'back', 'other', 'why', 'too', 'either', 'next', 'six', 'who', 'only', 'down', 'st

Stemming und Lemmatisierung

- Ziel: verschiedene Flexionsformen eines Wortes auf eine gemeinsame Basisform zurückführen
- **Stemming**: einfache, regelbasierte Verfahren, die Suffixe entfernen (z.B. `running` -> `run`, `happiness` -> `happi`). Oft werden nur die letzten paar Buchstaben abgeschnitten. Dies kann zu nicht existierenden Wörtern führen
- **Lemmatisierung**: komplexere Verfahren, die linguistische Informationen (z.B. Part-of-Speech) verwenden, um die Grundform (Lemma) eines Wortes zu bestimmen. Führt zu echten Wörtern (z.B. `better` -> `good`, `running` -> `run`)

```
# !pip install spacy
# !python -m spacy download de_core_news_sm
# !python -m spacy download en_core_web_sm
```

Lemmatisierung

```
import spacy
nlp_de = spacy.load('de_core_news_sm') # kleines deutschsprachiges Modell
nlp_en = spacy.load('en_core_web_sm') # kleines englischsprachiges Modell
```

```
docs = [nlp_de(i) for i in ["Pinguine", "schreibst", "geschrieben", "beste"]]
for d in docs:
    for token in d:
        print(f"{token} --> {token.lemma_}")
```

```
Pinguine --> Pinguine
schreibst --> schreibst
geschrieben --> schreiben
beste --> gut
```

```
doc = nlp_en("He is reading detective stories")
for token in doc:
    print(f"{token} --> {token.lemma_}")
```

```
He --> he
is --> be
reading --> read
detective --> detective
stories --> story
```



```
# Achten Sie auf das Token "meeting"
doc = nlp_en("Our meeting today was worse than yesterday, \
I'm scared of meeting the clients tomorrow")

for token in doc:
    print(f"{token} --> {token.lemma_}")
```

```
Our --> our
meeting --> meeting
today --> today
was --> be
worse --> bad
than --> than
yesterday --> yesterday
' --> '
I --> I
'm --> be
scared --> scared
of --> of
meeting --> meet
the --> the
clients --> client
tomorrow --> tomorrow
```

```
doc = nlp_de("Unser Treffen heute war schlimmer als gestern, \
Ich habe Angst davor, die Kunden morgen zu treffen")

for token in doc:
    print(f"{token} --> {token.lemma_}")
```

```
Unser --> unser
Treffen --> Treffen
heute --> heute
war --> sein
schlimmer --> schlimm
als --> als
gestern --> gestern
, --> --
Ich --> ich
habe --> haben
Angst --> Angst
davor --> davor
, --> --
die --> der
Kunden --> Kunde
morgen --> morgen
zu --> zu
treffen --> treffen
```

Wie funktioniert Lemmatisierung?

- Die effizientesten Methoden beinhalten eine vollständige morphologische Analyse jedes Wortes.
- Morphologie ist die Untersuchung der Art und Weise, wie Wörter aus kleineren bedeutungstragenden Einheiten, den sogenannten Morphemen, aufgebaut sind.
- Wir unterscheiden hierfür grob zwischen zwei Arten von Morphemen: das zentrale, Bedeutungstragende Morphem (Stamm) und Affixen, welche dran gehängt werden und nicht allein stehen können, und zusätzliche Bedeutung rein bringen. Z.B. Katzen → Katze (Stamm) + n (Affix)
- Es gibt verschiedene Arten von Affixen und sie können an verschiedenen Stellen im Wort stehen (vorn, hinten, beides)
- Es gibt verschiedene Lemmatisierungs-Verfahren, idR komplexe Algorithmen
- Typischerweise verwenden diese unter der Haube Part-of-Speech Tagging

- Lemmatizer in spaCy:
 - führe PoS durch
 - falls das Wort ein Nomen, Verb, Adjektiv oder Adverb ist, führe aus:
 - falls Wort in Dictionary für irreguläre Wörter, gebe Value zurück
 - sonst wende Suffix-Regel an

Stemming

- In der Regel heuristische Verfahren, bei denen Wort-Endungen abgeschnitten werden
- Beispiel: *fishing, fished, fisher* → *fish*
- Der Wortstamm muss selbst kein Wort sein, z.B. *argued, argues, arguing* → *argu*
- Geht schneller als Lemmatisierung, da z.B. kein PoS-tagging nötig
- Stemming ist wie Lemmatisierung ebenfalls Sprach-spezifisch

- Stemming: reduce to word stem
- Lemmatization: smartly reduce to word stem

"Our meeting today was worse than yesterday,
I'm scared of meeting the clients tomorrow."

Stemming:

```
['our', 'meet', 'today', 'wa', 'wors', 'than', 'yesterday', ',', 'i', "'m",  
'scare', 'of', 'meet', 'the', 'client', 'tomorrow', '.']
```

Lemmatization:

```
['our', 'meeting', 'today', 'be', 'bad', 'than', 'yesterday', ',', 'i', 'be',  
'scar', 'of', 'meet', 'the', 'client', 'tomorrow', '.']
```

- Es gibt verschiedene Stemmer, z.B. Porter, Snowball (Porter2), Lancaster. Unterschied in 'Aggressivität'
- Stemmer gibt es z.B. im Paket `nltk` aber nicht in `spaCy` - da nur Lemmatizer (ist eh besser).

Ähnlichkeit von Texten

Wort-Ähnlichkeiten (Word Similarity)

einfaches Beispiel: Synonyme

- Def.: Zwei Wörter sind synonym, wenn das eine durch das andere in jedem Satz ersetzt werden kann, ohne die Bedeutung des Satzes zu verändern.
- Beispiele?

- Couch / Sofa
 - Auto / Personenkraftwagen
 - sich übergeben / erbrechen
 - hübsch / schön
-
- Allerdings sind die Worte nie gleichbedeutend (gilt vermutlich für keine zwei Worte mit unterschiedlicher Wortform)
 - Synonyme sind häufig eher "annähernde Synonyme"
 - Und Worte haben i.A. nicht sehr viele Synonyme
-
- Katze und Hund sind sicher keine Synonyme aber in gewissem Sinne **ähnlich** (jedoch nicht im Sinne der Wortbedeutung wie bei Synonymen)
 - Wort-Ähnlichkeit spielt eine große Rolle in größeren semantischen NLP-Anwendungen, z.B. können sie verwendet werden, um die Ähnlichkeit der Bedeutung zweier Phrasen oder Sätze zu berechnen. Dies wiederum wird für NLP-Aufgaben wie Paraphrasierung, Question Answering oder Zusammenfassung (summarization) verwendet.
 - Es gibt verschiedene Formen von Wort-Ähnlichkeiten (Assoziation - **Kaffee** und **Tasse**, Semantisches Feld - **Skalpel**, **Arzt**, **Anästhesie** als Teil des Felds **Krankenhaus**, usw.) und wir kommen später (teilweise) darauf zurück.
 - Es gibt jedoch kein Modell, dass für alle Fälle perfekt passt. Das aktuell beste Modell heißt **Vektor-Semantik**.

Vektor-Semantik

- geht zurück auf die linguistische und philosophische Arbeit von Ludwig Wittgenstein aus den 50er Jahren. Er glaubte nicht daran, dass eine vollständig formale Theorie der Definition von Wort-Bedeutungen entwickelt werden kann, und schlug stattdessen vor, dass "die Bedeutung eines Wortes seine Verwendung in der Sprache ist". Das heißt, wir sollten Wörter dadurch definieren, wie sie von Menschen beim Sprechen und Verstehen

verwendet werden.

- Linguisten hatten bereits beobachtet, dass bspw. Synonyme wie **eye-doctor** und **oculist** (für "Augenarzt/ärztin") in sehr ähnlicher Umgebung auftauchten (z.B. nahe Worten wie **eye** oder **examined**), und dass die Ähnlichkeit der Begriffe "proportional" zu der Überschneidung ihrer Umgebungen war.
- Konkrete linguistische Idee zur Umsetzung der These von Wittgenstein: **Distributional Hypothesis** und lautet, vereinfacht gesagt, *a word is characterized by the company it keeps*.
- Formaler: wir definieren ein Wort durch seine Umgebung oder die **Verteilung im Sprachgebrauch**. Die Verteilung eines Wortes ist die Menge der Kontexte, in denen es vorkommt (die benachbarten Wörter oder grammatikalischen Umgebungen).

Beispiel

Angenommen, wir wissen nicht, was **Ong Choy** ist, aber wir finden es in folgenden Sätzen:

```
(S1) Ong Choy ist köstlich, wenn es mit Knoblauch gedünstet wird  
(S2) Ong Choy schmeckt vorzüglich auf Reis  
(S3) Ong Choy-Blätter mit salzigen Soßen ...
```

Außerdem, nehmen wir an, dass wir einige der Umgebungs-Wörter von **Ong Choy** gesehen haben in Kontexten wie:

```
... Spinat mit Knoblauch gedünstet und dazu Reis ...  
... Mangoldstängel und -blätter sind köstlich ...  
... Kohlgemüse und andere salzige Blattgemüse
```

Was können wir daraus über die Bedeutung von **Ong Choy** ableiten?

Ong Choy kommt vor mit Wörtern wie **Reis**, **Knoblauch**, **köstlich** und **salzig**, ebenso wie **Spinat**, **Mangold**

und Blattgemüse

→ suggeriert, dass Ong Choy ein Blattgrün ist, ähnlich wie diese anderen Blattgrünarten.

- Rechnerisch lässt sich das umsetzen, indem wir Wörter im Kontext von Ong Choy zählen. Wörter wie Spinat werden dann einen ähnlichen Vektor haben.

Wörter und Vektoren

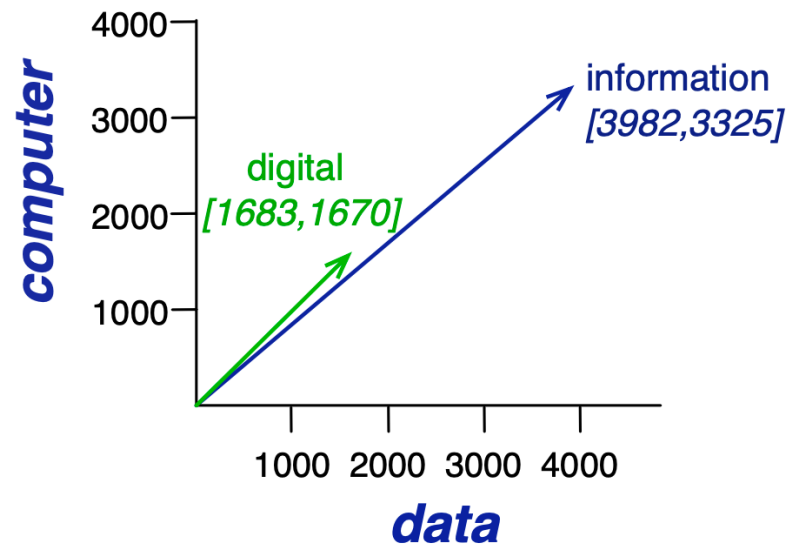
- Vector-Modelle (Eng.: auch distributional models) von Semantik basieren auf sog. Co-Occurrence Matrices.
- Eine solche haben wir bereits beim Bag-of-Words Ansatz kennengelernt: Dokument-Wort-Matrix (Englisch: **Document-Term-Matrix**).
- Zur Erinnerung: Diese war unser Vektor-Modell für Information Retrieval. Idee: zwei ähnliche Dokumente werden ähnliche Wörter beinhalten und somit werden ihre Wort-Vektoren ähnlich sein (wie auch immer das mathematisch operationalisiert wird).

			action	amazing	...	soldiers	thriller	...
		D1	2	1		2	0	
		D2	0	2		0	0	
		D3	0	0		0	2	

- Dokumente sind Vektoren bestehend aus Zahlen, welche die Häufigkeiten von Wörtern angeben
- jedes Wort repräsentiert eine Dimension des Vektorraums; Anzahl an Wörtern im Vokabular = Dimension des Vektorraums

- Wir können das Ganze umdrehen (technisch: Zeilen und Spalten vertauschen): Worte werden über Ihre Vorkommnisse in Dokumenten repräsentiert.
- Idee: Zwei Worte sind ähnlich, wenn sie ähnlich oft in ähnlichen Dokumenten vorkommen (**Document-Term-Matrix**)
- Weiterer Ansatz: **Term-Term-Matrix** oder **Word-Word-Matrix** oder **Term-Context-Matrix**
- Zeilen und Spalten repräsentieren jeweils die Wortformen im Korpus, d.h. Dimension der Matrix ist $|V| \times |V|$.
- Die Zellen geben an, wie häufig die Wörter im gleichen Kontext vorkommen. Kontext kann dabei sein: das gleiche Dokument, der gleiche Satz, maximal 4 Wörter auseinander, etc.
- Beispiel für letzteren Ansatz an Datenbeispielen aus Wikipedia:

is traditionally followed by	cherry	pie, a traditional dessert
often mixed, such as	strawberry	rhubarb pie. Apple pie
computer peripherals and personal	digital	assistants. These devices usually
a computer. This includes	information	available on the internet



Visualisierung der Wort-Vektoren für 'digital' und 'information' in einem 2D-Raum, aufgespannt durch die Vektoren von 'data' und 'computer'. Jurafsky & Martin, Speech and Language Processing (2026), Chapter

3

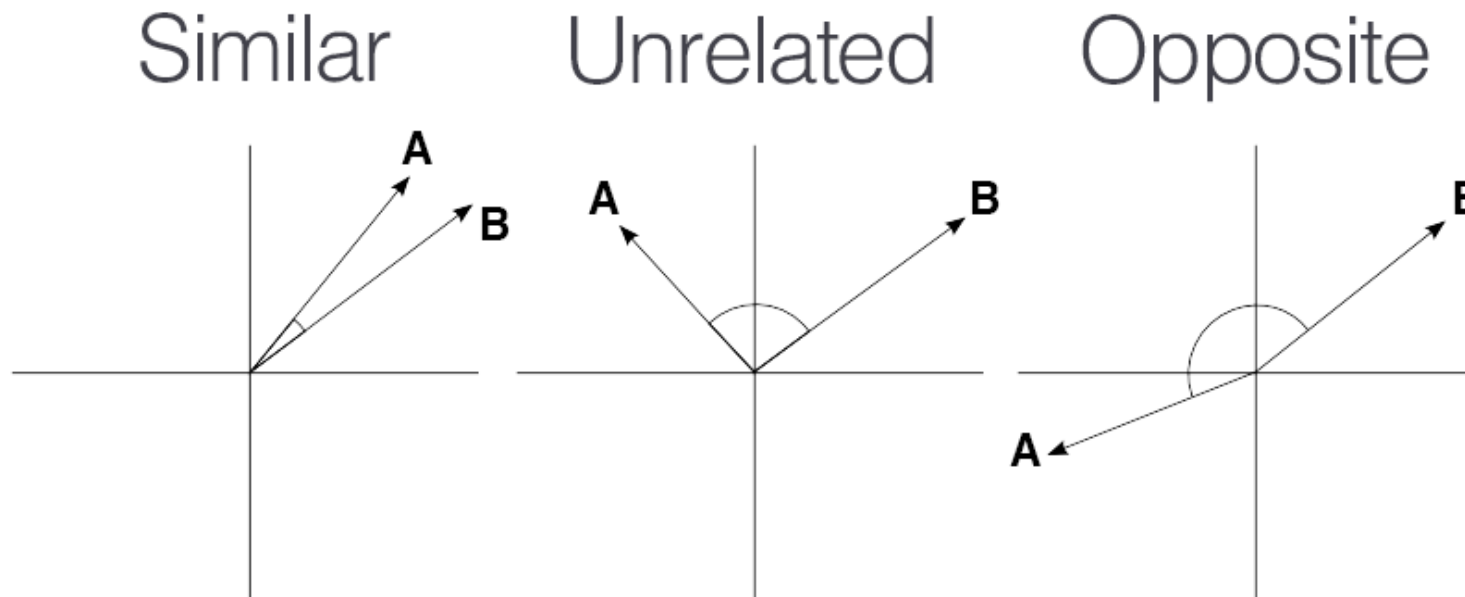
- Achtung: Wort-Vektoren haben normalerweise sehr viele Dimensionen
- Matrizen üblicherweise sparse (viele Nullen); es gibt effiziente Verfahren diese zu speichern und zu verarbeiten

Kosinus-Ähnlichkeit (Cosine similarity)

- am meisten verwendetes Maß für die Ähnlichkeit zweier Wörter v und w
- entspricht dem Kosinus des Winkels zwischen v und w
- Lässt sich über das Skalarprodukt berechnen:

$$\text{cosine}(v, w) := \frac{v \cdot w}{|v| \cdot |w|}$$

- Ausdruck im Nenner bewirkt eine Normalisierung auf Vektoren der Länge 1; ansonsten würden Wort-Vektoren mit hohen Einträgen und wenig Nullen (also Vektoren von Wörtern, die allgemein häufig vorkommen) immer sehr ähnlich sein, und das ist eher nicht hilfreich.
- Allgemein nimmt die Kosinus-Funktion Werte zwischen +1 (Vektoren zeigen in gleiche Richtungen) und -1 (unterschiedliche Richtungen); 0 wenn senkrecht zueinander. Aber: da Wort-Vektoren nur positive Einträge haben (wir zählen Vorkommnisse), erhalten wir Werte zwischen 0 und 1.



Interpretation der Kosinus-Ähnlichkeit

Statistische Text-Generierung

- Einfache Methode zur Text-Generierung basiert auf Shannon's Modell (1948)
- Grundidee: Texte bestehen aus Zeichen (Buchstaben, Satzzeichen, Leerzeichen, etc.). Wir können die Wahrscheinlichkeit eines Zeichens c gegeben den vorhergehenden n Zeichen $c_{i-n}, \dots, c_{i-1}$ modellieren:

$$P(c_i | c_{i-n}, \dots, c_{i-1})$$

- Für $n = 1$ haben wir ein **unabhängiges Modell**: $P(c_i | c_{i-1})$
- Für $n = 2$ haben wir ein sog. **Markov-Modell 1. Ordnung**: $P(c_i | c_{i-2}, c_{i-1})$
- Für $n = 3$ haben wir ein sog. **Markov-Modell 2. Ordnung**: $P(c_i | c_{i-3}, c_{i-2}, c_{i-1})$
- usw.

```
text = ""I felt happy because I saw the others were happy  
and because I knew I should feel happy, but I wasn't really happy.""
```

```
# Tokenisierung  
words = text.split()  
print(words)
```

```
['I', 'felt', 'happy', 'because', 'I', 'saw', 'the', 'others', 'were', 'happy', 'and', 'because', 'I']
```

```
# Bilde **Bigramme** (=Paare von aufeinanderfolgenden Wörtern):  
bigrams = []  
for i in range(len(words)-1):  
    bigrams.append( (words[i], words[i+1]) )  
print(bigrams)
```

```
[('I', 'felt'), ('felt', 'happy'), ('happy', 'because'), ('because', 'I'), ('I', 'saw'), ('saw', 'the
```

Verallgemeinerung hiervon sind N -Gramme:

- Unigramme (1-grams):

```
'I', 'felt', 'happy', 'because', 'I', 'saw', ...
```

- Bigramme (2-grams, bigrams):

```
'I felt', 'felt happy', 'happy because', 'because I', 'I saw', ...
```

- Trigramme (3-grams, trigrams):

```
'I felt happy', 'felt happy because', 'happy because I', 'because I saw', ...
```

- ...

```
# Berechne **Häufigkeiten** von Folgewörtern
from collections import defaultdict
bigram_freq = defaultdict(lambda: defaultdict(int))
for w1, w2 in bigrams:
    bigram_freq[w1][w2] += 1

# pretty print
for w1 in bigram_freq:
    print(f"{w1}: {dict(bigram_freq[w1])}")
```

```
I: {'felt': 1, 'saw': 1, 'knew': 1, 'should': 1, 'wasn't': 1}
felt: {'happy': 1}
happy: {'because': 1, 'and': 1}
because: {'I': 2}
saw: {'the': 1}
the: {'others': 1}
others: {'were': 1}
were: {'happy': 1}
and: {'because': 1}
knew: {'I': 1}
should: {'feel': 1}
feel: {'happy,': 1}
happy,: {'but': 1}
but: {'I': 1}
wasn't: {'really': 1}
really: {'happy.': 1}
```

Wir stellen uns nun vor, dass wir **einen großen Text-Corpus** zur Verfügung haben, z.B. die englischsprachige Wikipedia oder die Sammlung aller Shakespeare-Werke.

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Absolute Bigramm-Häufigkeiten für 8 von 1446 Wörtern aus dem 'Berkeley Restaurant Project' corpus von 9332 Sätzen. Jurafsky & Martin, Speech and Language Processing (2026), Chapter 3

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Relative Bigramm-Häufigkeiten für 8 von 1446 Wörtern aus dem 'Berkeley Restaurant Project' corpus von 9332 Sätzen. Jurafsky & Martin, Speech and Language Processing (2026), Chapter 3

Damit haben wir eine **Wahrscheinlichkeitsverteilung** für ein Wort basierend auf dem Wort davor:

$$P(\text{eat} \mid \text{to}) = 0.28$$

bedeutet, die Wahrscheinlichkeit für das Wort `eat` ist 0.28, wenn das Wort davor `to` ist (bedingte Wahrscheinlichkeiten).

Shannons Methode

`<s>` steht für den Satzanfang und `</s>` für das Satzende.

1. Start: Ziehe zufällig ein Bigramm `(<s>, w)` basierend auf der bedingten Wahrscheinlichkeit $P(w \mid <s>)$
2. Wiederhole: Ziehe zufällig ein Bigramm `(w, x)` basierend auf der bedingten Wahrscheinlichkeit $P(x \mid w)$
(erstes Wort im nächsten Bigramm soll gleich dem zweiten Wort im vorherigen Bigramm sein)
3. Ende: Ziehe zufällig ein Bigramm `(y, </s>)`

1 gram	Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives
2 gram	Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her
3 gram	They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

Drei zufällig generierte Sätze basierend auf N-Gramm Wahrscheinlichkeitsmodellen, welche aus 40 Millionen Wörtern aus dem Wall Street Journal berechnet wurden. Jurafsky & Martin, Speech and Language Processing (2026), Chapter 3

1 gram	—To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have —Hill he late speaks; or! a more to leg less first you enter
2 gram	—Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. —What means, sir. I confess she? then all sorts, he is trim, captain.
3 gram	—Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done. —This shall forbid it should be branded, if renown made it empty.
4 gram	—King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; —It cannot be but so.

Acht zufällig generierte Sätze basierend auf dem Shakespeare-Corpus. Jurafsky & Martin,
Speech and Language Processing (2026), Chapter 3